

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO
FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
ESCUELA DE INGENIERÍA EN SISTEMAS

Diseño de la Base de Datos

BioMove App

SQLite (desarrollo) / PostgreSQL (producción) — SQLAlchemy async + aiosqlite

Versión:	2.0 — Final
Fecha:	13 de junio de 2026
Autor:	Bryan Xavier Calderón Novillo
Institución:	ESPOCH — APLICACIONES 2 — Ing. Sistemas
SGBD desarrollo:	SQLite 3.x con aiosqlite (async)
SGBD producción:	PostgreSQL 15.x (mismo ORM SQLAlchemy)
ORM:	SQLAlchemy 2.x async (AsyncSessionLocal)

1. Introducción

⚠ CORRECCIÓN CRÍTICA (Versión 2.0): El documento de base de datos versión 1.0 describía Cloud Firestore (NoSQL) y Firebase Storage como tecnologías de almacenamiento. Ambas son INCORRECTAS. El sistema BioMove implementa una base de datos RELACIONAL: SQLite (desarrollo) y PostgreSQL (producción) mediante SQLAlchemy 2.x async. Firebase Storage NO se usa para videos. Firebase Auth se usa ÚNICAMENTE para autenticación JWT.

Este documento presenta el diseño de la base de datos relacional de BioMove App, un aplicativo móvil de análisis biomecánico de sentadilla. La base de datos almacena: perfiles de usuarios (athletes, coaches, admins), trabajos de análisis de video (VideoJob), sesiones de entrenamiento con 40+ parámetros biomecánicos por repetición (TrainingSession, Repetition), retroalimentación automática (FeedbackItem), modelos IA personales (UserAIModel con EMA $\alpha=0.08$), récords de fuerza (StrengthRecord), mejores/peores repeticiones (BestRep), planes de progresión VBT de 11 semanas (TrainingPlan), mensajes de chat coach-atleta (ChatMessage), planes de mejora generados por IA (ImprovementPlan) y vínculos coach-atleta (CoachAthleteLink).

2. Objetivos del diseño

- Definir la estructura relacional completa para soportar: registro de usuarios (roles: athlete/coach/admin), análisis biomecánico (40+ parámetros por rep), flujo coach-atleta, VBT, chat y modelo IA personal.
- Garantizar integridad referencial mediante claves foráneas (FK) entre todas las entidades relacionadas.
- Facilitar la integración con el backend FastAPI/Celery mediante SQLAlchemy async (AsyncSessionLocal, aioredis).
- Soportar migración transparente de SQLite (desarrollo) a PostgreSQL (producción) sin cambios en el código ORM.
- Asegurar la seguridad de acceso: todos los datos son accesibles solo mediante JWT Firebase verificado en FastAPI.

3. Consideraciones técnicas

3.1 Motor de base de datos

Aspecto	Desarrollo (SQLite)	Producción (PostgreSQL)
Motor	SQLite 3.x (aiosqlite)	PostgreSQL 15.x
Driver async	aiosqlite	asyncpg
Conexión	sqlite+aiosqlite:///./biomove.db	postgresql+asyncpg://user:pass@host/db
Concurrencia	1 escritor simultáneo	Múltiples lectores/escritores concurrentes
Instalación	Sin servidor (archivo local)	Servidor PostgreSQL dedicado
Migraciones	Scripts SQL manuales (sin Alembic)	Alembic recomendado en producción
Uso en BioMove	Desarrollo local, testing	Despliegue final de la tesis

3.2 ORM: SQLAlchemy 2.x async

- AsyncSessionLocal: sesión asíncrona creada con AsyncSession(engine).
- create_async_engine(): motor con SQLite (aiosqlite) o PostgreSQL (asyncpg).

- `create_tables()`: crea todas las tablas al startup de FastAPI (app lifespan).
- Migraciones: manuales en SQLite (ALTER TABLE directo). Sin Alembic en .
- Bug corregido: `shared_with_coach` requirió ALTER TABLE manual: `ALTER TABLE training_sessions ADD COLUMN shared_with_coach BOOLEAN DEFAULT FALSE;`
- Bug corregido: `VideoView` ENUM solo acepta ("lateral", "frontal", "both"). El valor "auto" causaba SQLAlchemy LookupError.

3.3 Autenticación y seguridad

- `firebase_auth (JWT)`: todos los endpoints FastAPI requieren Bearer token válido en Authorization header.
- `verify_firebase_token()`: función de verificación con `firebase_admin`. Retorna uid (`firebase_uid`) del usuario.
- `get_current_user` dependency: decorator aplicado a todos los endpoints protegidos del backend.
- Almacenamiento de videos: LOCAL en servidor FastAPI (`./uploads/` y `./exports/annotated/`). NO usa Firebase Storage.
- IDs: todos los modelos usan UUID como clave primaria (no autoincrement entero).

4. Modelo de datos: Tablas SQLAlchemy (SQLite/PostgreSQL)

A continuación se describe cada tabla del modelo relacional. Leyenda:  PK = Clave Primaria (UUID),  FK = Clave Foránea, ENUM = Tipo enumerado (validado por SQLAlchemy).

Tabla: users

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID (str)	Identificador único del usuario	NO	PK
firebase_uid	VARCHAR UNIQUE NOT NULL	UID de Firebase Auth — usado para lookup JWT	NO	UNIQUE
email	VARCHAR UNIQUE NOT NULL	Correo electrónico del usuario	NO	UNIQUE
role	ENUM("athlete","coach","admin")	Rol en el sistema. Determina acceso y dashboard	NO	ENUM CHECK
height_cm	FLOAT	Altura del usuario en centímetros (ingresada en onboarding)	SÍ	
weight_kg	FLOAT	Peso en kilogramos (enviado como double en FormData de Dio)	SÍ	
age	INTEGER	Edad del usuario	SÍ	
sex	VARCHAR(10)	Sexo biológico (M/F/otro) — relevante para cálculos biomecánicos	SÍ	
training_years	FLOAT	Años de experiencia en entrenamiento de fuerza	SÍ	
onboarding_done	BOOLEAN DEFAULT FALSE	Si el usuario completó el proceso de onboarding. Cacheado en SharedPreferences.	NO	DEFAULT FALSE

created_at	TIMESTAMP DEFAULT NOW()	Fecha y hora de registro	NO	DEFAULT NOW()
-------------------	-------------------------	--------------------------	----	---------------

Notas: firebase_uid se obtiene del JWT verificado con verify_firebase_token(). La contraseña NO se almacena (Firebase Auth la gestiona). weight_kg debe enviarse como double (no String) en FormData de Dio para evitar error 422 de FastAPI/Pydantic.

Tabla: video_jobs

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único del trabajo de análisis (job_id)	NO	PK
user_id	UUID	FK → users.id. Usuario propietario del video	NO	FK → users.id
status	ENUM("pending", "processing", "completed", "failed")	Estado del análisis en la cola Celery	NO	ENUM CHECK
progress_pct	INTEGER DEFAULT 0	Porcentaje de progreso del análisis (0-100%). Actualizado cada 5% por Celery.	NO	0-100
annotated_download_url	VARCHAR	URL relativa del video anotado (/exports/annotated/...mp4). Frontend agrega baseUrl.	Sí	
error_message	TEXT	Mensaje de error si status="failed". Incluye stack trace resumido.	Sí	
created_at	TIMESTAMP DEFAULT NOW()	Fecha y hora de inicio del trabajo	NO	DEFAULT NOW()
updated_at	TIMESTAMP	Última actualización del status/progress_pct	Sí	ON UPDATE

Notas: La URL annotated_download_url es RELATIVA (no incluye el host/puerto). En Flutter, la URL completa es: AppConstants.baseUrl + annotated_download_url. El frontend hace polling GET /videos/{job_id}/status cada 3 segundos.

Tabla: training_sessions

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único de la sesión de entrenamiento	NO	PK
user_id	UUID	FK → users.id. Atleta propietario de la sesión	NO	FK → users.id
job_id	UUID	FK → video_jobs.id. Trabajo de análisis asociado	NO	FK → video_jobs.id
exercise_type	VARCHAR DEFAULT "squat"	Tipo de ejercicio analizado (solo "squat" en)	NO	DEFAULT "squat"
video_view	ENUM("lateral", "frontal", "both")	Vista del video detectada automáticamente. SOLO estos 3 valores (no "auto").	NO	ENUM CHECK
technique_score	FLOAT	Score global de técnica 0-100	Sí	0.0-100.0

		(promedio de rep_scores de todas las repeticiones)		
total_reps	INTEGER	Número total de repeticiones detectadas por ángulo de rodilla	SÍ	≥0
fatigue_detected	BOOLEAN DEFAULT FALSE	True si variación en score de últimas reps >20% (indicador de fatiga)	NO	DEFAULT FALSE
shared_with_coach	BOOLEAN DEFAULT FALSE	Si el atleta ha compartido esta sesión explícitamente con su coach	NO	DEFAULT FALSE — migración manual
created_at	TIMESTAMP DEFAULT NOW()	Fecha y hora de la sesión	NO	DEFAULT NOW()

Tabla: repetitions

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único de la repetición	NO	PK
session_id	UUID	FK → training_sessions.id	NO	FK → training_sessions.id
rep_number	INTEGER	Número de repetición en la serie (1, 2, 3...N)	NO	≥1
rep_score	FLOAT	Score de la repetición (0-100). Calculado por clasificador + umbrales.	SÍ	0.0-100.0
knee_angle_min	FLOAT	Ángulo mínimo de rodilla en grados (profundidad de sentadilla). Umbral ≤95° para buena profundidad.	SÍ	grados
left_valgus	FLOAT	Valgus de rodilla izquierda en grados. Umbral seguro <10°.	SÍ	grados
right_valgus	FLOAT	Valgus de rodilla derecha en grados. Umbral seguro <10°.	SÍ	grados
trunk_lean_max	FLOAT	Inclinación máxima del tronco hacia adelante en grados. Umbral <55°.	SÍ	grados
acl_risk_score	FLOAT	Score de riesgo de lesión ACL (0-100). Basado en valgus + velocidad de rodilla.	SÍ	0-100
lumbar_risk_score	FLOAT	Score de riesgo lumbar (0-100). Basado en butt_wink + inclinación.	SÍ	0-100
patellofemoral_load	FLOAT	Carga patelofemoral estimada (0-100). Basado en ángulo rodilla + velocidad.	SÍ	0-100
eccentric_duration_s	FLOAT	Duración de la fase excéntrica en segundos. Rango óptimo: 2-3s.	SÍ	segundos
concentric_duration_s	FLOAT	Duración de la fase concéntrica en segundos.	SÍ	segundos
bar_velocity_ms	FLOAT	Velocidad de barra estimada en metros/segundo (VBT). Calculado por desplazamiento de cadera.	SÍ	m/s
knee_asymmetry_pct	FLOAT	Asimetría entre rodilla izquierda y derecha en %. Umbral seguro <8%.	SÍ	%
depth_achieved	BOOLEAN	True si la rodilla alcanzó ≤95° (profundidad mínima de sentadilla paralela).	SÍ	booleano
clf_class	VARCHAR	Clase asignada por el clasificador scikit-learn: excelente/buena/aceptable/mejoras/riesgo	SÍ	5 valores

clf_confidence	FLOAT	Confianza del clasificador (0-1). Útil para detectar casos ambiguos.	SÍ	0.0-1.0
-----------------------	-------	--	----	---------

Tabla: feedback_items

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único del feedback item	NO	PK
session_id	UUID	FK → training_sessions.id	NO	FK → training_sessions.id
error_type	VARCHAR	Tipo de error detectado (ej: "valgus_knee", "insufficient_depth", "trunk_lean_excessive")	NO	
severity	ENUM("low","medium","high")	Nivel de severidad del error. "high" indica riesgo de lesión.	NO	ENUM CHECK
message	TEXT	Mensaje descriptivo del error para el usuario (ej: "Se detectó valgo de rodilla izquierda")	NO	
correction	TEXT	Corrección recomendada (ej: "Mantén las rodillas alineadas con los pies")	SÍ	
frequency	INTEGER	Número de repeticiones en las que ocurrió este error	NO	≥0
frequency_pct	FLOAT	Porcentaje de repeticiones con este error (frequency/total_reps × 100)	SÍ	0.0-100.0
is_injury_risk	BOOLEAN DEFAULT FALSE	True si este error representa un riesgo de lesión significativo	NO	DEFAULT FALSE

Tabla: user_ai_models

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único del modelo IA personal	NO	PK
user_id	UUID UNIQUE	FK → users.id. Relación 1:1 (un modelo por usuario)	NO	FK → users.id, UNIQUE
phase	ENUM("collecting","learning","active","optimizing")	Fase actual del modelo. Determina tipo de recomendaciones disponibles.	NO	ENUM CHECK
total_reps	INTEGER DEFAULT 0	Total de repeticiones procesadas por el modelo EMA	NO	DEFAULT 0
baseline_reps	INTEGER DEFAULT 10	Número de reps base para establecer el baseline personal (mínimo 10, ≤50% 1RM)	NO	DEFAULT 10
params_model	JSON	Valores EMA actuales de 18 parámetros clave. Formato: {"knee_angle_min": valor_ema, "left_valgus": ..., ...}	SÍ	JSON dict
morphology	JSON	Morfología detectada del atleta. Formato: {"long_femur": bool, "ankle_mobility": str, ...}	SÍ	JSON dict
improvement_trends	JSON	Tendencias de mejora/empeoramiento. Formato: {"knee_angle": "improving", "valgus": "stable", ...}	SÍ	JSON dict
updated_at	TIMESTAMP	Última vez que el modelo fue actualizado (post-análisis en Celery)	SÍ	ON UPDATE

Tabla: strength_records

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único del récord de fuerza	NO	PK
user_id	UUID	FK → users.id	NO	FK → users.id
exercise_type	VARCHAR DEFAULT "squat"	Ejercicio del récord	NO	DEFAULT "squat"
weight_kg	FLOAT	Peso usado en kilogramos	NO	>0
reps	INTEGER	Repeticiones realizadas con ese peso	NO	≥1
one_rm	FLOAT	1RM estimado en kg (Epley: $w \times (1 + r/30)$ o Brzycki: $w / (1.0278 - 0.0278r)$)	SÍ	>0

velocity_ms	FLOAT	Velocidad de barra en m/s (si disponible, para cálculo VBT más preciso)	SÍ	m/s
recorded_at	TIMESTAMP DEFAULT NOW()	Fecha y hora del registro	NO	DEFAULT NOW()

Tabla: best_reps

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único del mejor/peor rep	NO	PK
user_id	UUID	FK → users.id	NO	FK → users.id
exercise_type	VARCHAR DEFAULT "squat"	Ejercicio de referencia	NO	DEFAULT "squat"
rep_type	ENUM("best", "worst")	Tipo: mejor (mayor rep_score) o peor (menor rep_score) rep histórica	NO	ENUM CHECK
rep_score	FLOAT	Score de la repetición almacenada (0-100)	NO	0.0-100.0
clip_url	VARCHAR	URL relativa del clip de video de esa repetición (en FastAPI StaticFiles)	SÍ	URL relativa
params_json	JSON	Los 40+ parámetros biomecánicos de esa repetición específica	SÍ	JSON dict
updated_at	TIMESTAMP	Última vez que se actualizó (update_best_rep en tasks.py post-análisis)	NO	ON UPDATE

Tabla: training_plans

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único del plan de entrenamiento	NO	PK
user_id	UUID	FK → users.id. Atleta propietario del plan	NO	FK → users.id
exercise_type	VARCHAR DEFAULT "squat"	Ejercicio del plan	NO	DEFAULT "squat"
current_1rm	FLOAT	1RM actual del atleta (kg) usado para calcular el plan	NO	>0
block_type	ENUM("accumulation","intensification","realization","deload")	Bloque actual de entrenamiento	NO	ENUM CHECK
block_week	INTEGER	Semana actual dentro del bloque (1-4)	NO	1-4
plan_data	JSON	Plan completo 11 semanas con sesiones, series, reps, % 1RM y velocidades objetivo. Generado por progression_calculator.py	NO	JSON dict complejo
coach_approved	BOOLEAN DEFAULT FALSE	Si el coach ha aprobado este plan de entrenamiento	NO	DEFAULT FALSE
created_at	TIMESTAMP DEFAULT NOW()	Fecha de creación del plan	NO	DEFAULT NOW()

Tabla: chat_messages

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único del mensaje	NO	PK
sender_id	UUID	FK → users.id. Usuario que envió el mensaje	NO	FK → users.id
receiver_id	UUID	FK → users.id. Usuario destinatario del mensaje	NO	FK → users.id
message	TEXT	Contenido del mensaje de texto	SÍ	(null si es solo referencia)
msg_type	ENUM("text","analysis_ref","video_ref")	Tipo de mensaje. analysis_ref incluye ref_id de sesión; video_ref incluye ref_id de job.	NO	ENUM CHECK
ref_id	UUID	ID de referencia: session_id (analysis_ref) o job_id (video_ref)	SÍ	UUID nullable
is_read	BOOLEAN DEFAULT FALSE	Si el destinatario ha leído el mensaje	NO	DEFAULT FALSE
sent_at	TIMESTAMP DEFAULT NOW()	Fecha y hora de envío	NO	DEFAULT NOW()

Tabla: improvement_plans

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único del plan de mejora	NO	PK
user_id	UUID	FK → users.id. Atleta receptor del plan	NO	FK → users.id
session_id	UUID	FK → training_sessions.id. Sesión de origen del plan	SÍ	FK → training_sessions.id
plan_data	JSON	Plan de mejora generado: objetivos, ejercicios correctivos, cronograma. Generado por IA o coach.	NO	JSON dict
ai_generated	BOOLEAN DEFAULT TRUE	True si fue generado automáticamente por el sistema; False si fue creado por el coach.	NO	DEFAULT TRUE
created_at	TIMESTAMP DEFAULT NOW()	Fecha de creación del plan de mejora	NO	DEFAULT NOW()

Tabla: coach_athlete_links

Campo	Tipo de dato	Descripción	Nullable	Restricción
id	UUID	Identificador único del vínculo	NO	PK
coach_id	UUID	FK → users.id. Usuario con role="coach"	NO	FK → users.id
athlete_id	UUID	FK → users.id. Usuario con role="athlete"	NO	FK → users.id
status	ENUM("pending","active","inactive")	Estado del vínculo. "pending"=solicitud enviada, "active"=vínculo activo.	NO	ENUM CHECK
linked_at	TIMESTAMP DEFAULT NOW()	Fecha y hora de activación del vínculo (status="active")	SI	DEFAULT NOW()

5. Diagrama Entidad-Relación



6. Relaciones entre entidades

Entidad origen	Entidad destino	Tipo	Descripción de la relación
users	video_jobs	1:N	Un usuario puede tener múltiples trabajos de análisis de video
users	training_sessions	1:N	Un usuario puede tener múltiples sesiones de entrenamiento
users	user_ai_models	1:1	Cada usuario tiene un único modelo IA personal (EMA)
users	best_reps	1:N	Un usuario puede tener múltiples mejores/peores reps por ejercicio
users	strength_records	1:N	Un usuario puede tener múltiples récords de fuerza
users	training_plans	1:N	Un usuario puede tener múltiples planes de progresión
users	improvement_plans	1:N	Un usuario puede tener múltiples planes de mejora
users	chat_messages (sender)	1:N	Un usuario puede enviar múltiples mensajes de chat
users	chat_messages (receiver)	1:N	Un usuario puede recibir múltiples mensajes de chat
users (coach)	coach_athlete_links	1:N	Un coach puede vincularse con múltiples atletas
users (athlete)	coach_athlete_links	1:N	Un atleta puede tener múltiples solicitudes de vínculo (con diferentes coaches)
video_jobs	training_sessions	1:1	Un trabajo de análisis genera exactamente una sesión de entrenamiento

training_sessions	repetitions	1:N	Una sesión contiene múltiples repeticiones (con 40+ params cada una)
training_sessions	feedback_items	1:N	Una sesión genera múltiples ítems de retroalimentación
training_sessions	improvement_plans	1:N	Una sesión puede generar múltiples planes de mejora

7. Flujo de integración con el backend (FastAPI + Celery)

#	Paso del flujo de integración
1	Flutter graba/selecciona video. Sube vía Dio multipart POST /videos/upload-and-analyze (sin Content-Type manual, weight_kg como double).
2	FastAPI recibe el video. Crea VideoJob (status="pending", progress_pct=0) en SQLite. Almacena el video en ./uploads/{user_id}/{job_id}.mp4. Retorna {job_id}.
3	FastAPI encola la tarea en Celery (cola "normal" para atletas). El worker Celery recibe la tarea del broker Redis (puerto 6379).
4	Celery worker ejecuta el análisis: MediaPipe Pose (model_complexity=2) detecta 33 keypoints en frames impares. Frames pares → cv2.addWeighted(0.45/0.55). Auto-detección lateral/frontal por ancho de hombros.
5	squat_analyzer_40.py calcula 40+ parámetros por repetición. Máquina de estados de ángulo de rodilla detecta y cuenta reps. Umbrales biomecánicos aplicados.
6	classifier.py clasifica cada rep (scikit-learn .pkl, 43 features, 5 clases). Fallback a reglas fijas si no hay .pkl. personal_model.py actualiza EMA $\alpha=0.08$.
7	Celery actualiza VideoJob.progress_pct cada 5% durante el procesamiento.
8	video_annotator.py genera video anotado. Almacena en ./exports/annotated/{job_id}_annotated.mp4 (FastAPI StaticFiles).
9	Celery crea TrainingSession, Repetitions (40+ params), FeedbackItems en SQLite. update_best_rep() actualiza BestRep si hay nueva mejor/peor rep. Actualiza VideoJob.status="completed".
10	Flutter detecta status="completed" (polling cada 3s). Ejecuta GET /videos/{job_id}/results. ResultsScreen muestra: ScoreGauge, VideoPlayer (URL=baseUrl+annotated_url), FeedbackItems, tabs de parámetros.

8. Justificación técnica de las decisiones de diseño

Base de datos relacional (SQLite/PostgreSQL) en lugar de Firestore: El modelo de datos de BioMove es intrínsecamente relacional: User 1:N TrainingSession 1:N Repetition, con FKs entre 12 entidades. Firestore (NoSQL orientado a documentos) no soporta JOINS eficientes ni integridad referencial. El diseño relacional garantiza: (1) consistencia de datos mediante FK constraints, (2) consultas complejas (ej: obtener todas las sessions de un atleta vinculado a un coach con shared_with_coach=True) sin denormalización, (3) migración transparente de SQLite (dev) a PostgreSQL (prod) sin cambiar el código ORM.

UUID como clave primaria en todas las tablas: Se usan UUIDs (str) en lugar de INTEGER autoincrement para: (1) evitar colisiones al migrar datos entre SQLite y PostgreSQL, (2) mayor seguridad (los IDs no son predecibles/secuenciales), (3) compatibilidad con firebase_uid (que también es un string no secuencial), (4) facilitar futura distribución/sharding.

JSON para datos variables (params_model, plan_data, params_json, morphology): Campos JSON se usan para datos cuya estructura varía o tiene muchos campos: (1) UserAIModel.params_model: 18 valores EMA variables según el atleta, (2) BestRep.params_json: los 40+ parámetros de una rep específica, (3) TrainingPlan.plan_data: plan completo de 11 semanas con estructura compleja. SQLite soporta JSON nativo; PostgreSQL tiene soporte JSONB con índices.

ENUM para campos con valores acotados: Los campos ENUM garantizan integridad de datos a nivel de base de datos: role ("athlete","coach","admin"), video_view ("lateral","frontal","both" — NOT "auto", bug corregido), status ("pending","processing","completed","failed"), phase ("collecting","learning","active","optimizing"), rep_type ("best","worst"), severity ("low","medium","high"), msg_type ("text","analysis_ref","video_ref"), block_type ("accumulation","intensification","realization","deload"), link_status ("pending","active","inactive").

FastAPI StaticFiles para servir videos (NO Firebase Storage): Los videos se almacenan localmente y se sirven por FastAPI StaticFiles: (1) elimina dependencia y costo de Firebase Storage, (2) mayor velocidad de acceso (mismo servidor que la API), (3) control total sobre la retención y acceso de videos por user_id, (4) URLs: ./uploads/{user_id}/{job_id}.mp4 (originales) y ./exports/annotated/{job_id}_annotated.mp4 (anotados). La URL retornada por el backend es relativa; Flutter debe agregar el prefijo AppConstants.baseUrl.

shared_with_coach como BOOLEAN en TrainingSession: En lugar de una tabla de permisos separada, se usa un simple campo BOOLEAN en TrainingSession: (1) simplicidad de implementación (consulta directa WHERE shared_with_coach=TRUE), (2) el atleta tiene control granular (puede compartir sesiones específicas, no todas), (3) el valor por defecto FALSE garantiza privacidad. La columna fue añadida mediante migración manual (ALTER TABLE) en SQLite.